

CLAUDE CODE × SERVICENOW SDK

Video 5: Global-Scope Apps

When and How to Build Outside a Scoped Application

By Tushar | ReAcademy.ai | March 2026

Prerequisites

- Videos 1–4 completed
- Familiarity with ServiceNow scoped vs. global scope concepts

When to Use Global Scope

Scoped applications (x_vendor_app prefix) should be your default. Only go global when you must:

- Extending out-of-box tables (incident, change_request, cmdb_ci, etc.) with new fields
- Building integrations that must interact across multiple app scopes
- Creating platform-level utilities that all applications need
- Working with system properties or platform configurations

WARNING: Global scope is more powerful but also more dangerous. Global business rules run for ALL records, global ACLs affect ALL users. Always add conditions and test thoroughly.

Step 1 — Create a Global Application Record

Create a Global application record on your instance or use ServiceNow SDK to build it from terminal

1. Log into your instance as admin
2. Navigate to System Applications → Application Manager
3. Click “Create New”
4. Set Scope to Global
5. Give it a name (e.g., “Global Integration Utils”)
6. Save the record and note the sys_id — you’ll need it for now.config.json

TIP: Always create a dedicated application record. Don't dump everything into the default Global scope — it makes tracking and cleanup very difficult.

Step 2 — Check now.config.json

The key difference from scoped apps is in the config file.

Scoped app (what you've been doing):

```
{
  "scope": "x_acme_myapp",
  "scopeId": "<sys_id_of_scoped_app>"
}
```

Global app:

```
{
  "scope": "global",
  "scopeId": "<sys_id_of_your_global_app_record>"
}
```

The scopeId points to the Global application record you created in Step 1.

Step 3 — Naming Conventions in Global Scope

In scoped apps, everything uses the x_vendor_app_ prefix. In global scope, there is no automatic prefix, but you should still use a consistent convention to avoid name collisions.

Recommended prefixes:

Element	Convention	Example
Custom fields on OOB tables	u_ prefix	u_external_ref_id
New global tables	u_ or descriptive prefix	u_integration_log

Script includes	Descriptive class name	IntegrationHelper
Business rules	Descriptive name	Clear Retired Asset Assignee

Step 4 — Fetch Type Definitions

When extending OOB tables, you need the existing table schemas so the Fluent compiler knows about their columns.

```
now-sdk dependencies --auth mydev
```

This fetches type definitions for tables like incident, change_request, cmdb_ci, etc. into the @types/ folder. Claude Code can then use these for accurate code generation.

Step 5 — Generate Code with Claude Code

```
claude
```

Be explicit about global scope in your prompt. Example:

```
This is a global-scope app (scope: global in now.config.json).
```

```
Use the u_ prefix for custom fields.
```

```
Add a new field called u_external_ref_id (string, 100 chars)
```

```
if the short_description contains 'EXTERNAL', auto-populate
```

`u_external_ref_id` by extracting the text between brackets.

TIP: The `sndev` skill defaults to scoped-app patterns. Always tell Claude “this is global scope” and “use `u_` prefix” so it generates the correct naming.

Step 6 — Build & Deploy

```
now-sdk build
```

```
now-sdk install --auth mydev
```

Verify on your instance:

1. Go to Application Manager — your app appears under Global scope
2. Navigate to the extended table (e.g., Incident)
3. Open a record — your custom field should be visible
4. Test any business rules by creating/updating records

Global Scope Best Practices

Do	Don't
Create a dedicated Global app record	Dump everything into default Global
Always add conditions to business rules	Create unconditional rules on OOB tables
Use script includes for reusable logic	Write global functions (pollutes namespace)
Test ACLs carefully before deploying	Apply broad ACLs without testing
Document all global customizations	Rely on memory for what you changed
Use <code>u_</code> prefix consistently	Mix naming conventions